C++プログラミングI

- 第6回：vectorによる多次元配列
- 担当：二瓶芙巳雄

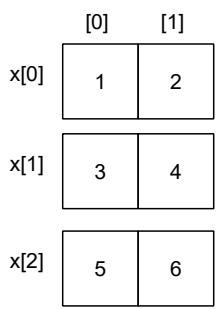
多次元配列とは

- 一次元の情報：
  - 一つの座標で表す長さなどの情報
- 多次元の情報：
  - 絵の情報はxy座標上の点の集まり（2次元）
  - 動画の情報はxyとフレーム番号（3次元）
- **C++による多次元情報の扱い**
  - 1次元配列：配列
  - 2次元配列：**配列を要素とする配列**
  - 3次元配列：2次元配列を要素とする配列
- `vector`の2次元配列
  - `vector<T>`を要素とする`vector`
  - ※`<T>`の`T`(Template)は任意の型．`int`や`double`など．

```
1   std::vector<std::vector<T>> x;
```

`vector`の2次元配列

- 2次元配列：1次元配列の配列
  - 1次元`int`配列：`vector<int>`
  - 2次元`int`配列：`vector< vector<int> >`
- `vector<vector<int>> x;`は要素数0
  - ※ 配列の入れ物だけ作ったイメージ
- 追加後の`x`と`a`, `b`, `c`は無関係




```cpp
// 3x2の2次元配列
using std::cout, std::vector;

int main() {
  vector<vector<int>> x;
  vector a{1,2}, b{3,4}, c{5,6};

  x.push_back(a);
  x.push_back(b);
  x.push_back(c);

  cout << x[0][0] <<" "<< x[0][1] <<"\n";// 1 2
  cout << x[1][0] <<" "<< x[1][1] <<"\n";// 3 4
  cout << x[2][0] <<" "<< x[2][1] <<"\n";// 5 6
}
```

```
% ./a.out
1 2
3 4
5 6
```

2次元配列へのアクセス

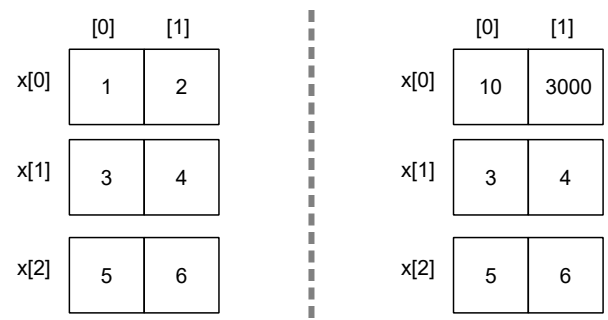
- `x[0]` や `x[1]` は1次元配列
- `x[0][0]` や `x[1][0]` は `int`
- `x[i][j]` は左辺値として指定できる

```
1   int main() {
2     vector<vector<int>> x;
3     vector a{1,2}, b{3,4}, c{5,6};
4     x.push_back(a); x.push_back(b); x.push_back(c);
5
6     cout << x[0][0] <<" "<< x[0][1] <<"\n";// 1 2
7     cout << x[1][0] <<" "<< x[1][1] <<"\n";// 3 4
8     cout << x[2][0] <<" "<< x[2][1] <<"\n";// 5 6
9     cout << "\n";
10
11    x[0][0] = 10; // 先頭要素を変更
12    std::cin >> x[0][1]; // [0][1] 番目の要素に入力
13    cout << "\n";
14
15    cout << x[0][0] <<" "<< x[0][1] <<"\n";// 1 2
16    cout << x[1][0] <<" "<< x[1][1] <<"\n";// 3 4
17    cout << x[2][0] <<" "<< x[2][1] <<"\n";// 5 6
18  }
```

```
% ./a.out
1 2
3 4
5 6

3000

10 3000
3 4
5 6
```

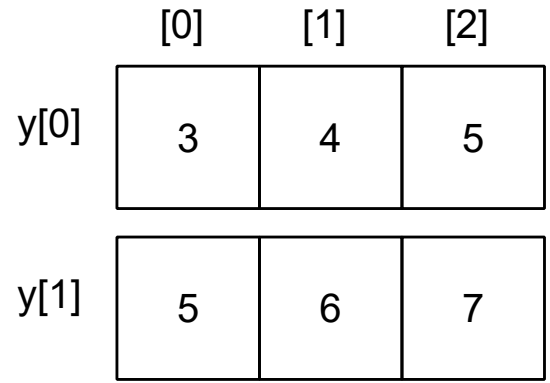


2次元配列の初期化

- 中括弧 `{}` による初期化リストを並べる

```
vector<vector<int>> y {{3,4,5},{5,6,7}};

// 改行するとわかりやすい
vector<vector<int>> y {
  {3,4,5},
  {5,6,7}
};
```



- 要素がすべて同じ場合，丸括弧 `()` を使う
  - 50x100の2次元配列, 値は1

```
vector<vector<int>> z(50, vector<int>(100, 1));
または
vector z(50, vector(100, 1)); // 省略形
```

1次元配列と2次元配列の初期化の比較

```cpp
// 1次元配列の初期化

// 要素なし
vector<double> a;

// 値の直接指定
vector<double> b {1.0, 1.1, 1.2};




// 要素数5, 初期値が0.0
vector<double> c(5);

// 要素数5, 初期値1.4
vector<double> d(5, 1.4);

// 以下も可
vector<int> z { vector<int>(3, 1) };
vector<int> y { vector(3, 1) };
```

```cpp
// 2次元配列の初期化

// 要素なし
vector<vector<double>> a;

// 値の直接指定：2x3
vector<vector<double>> b {
  {1.0, 1.1, 1.2},
  {2.0, 2.1, 2.2}
};

// 要素数5, 初期値が要素なしvector<double>：5x0
vector<vector<double>> c(5);

// 要素数5, 初期値が要素数30値1.4のvector<double>：5x30
vector<vector<double>> d(5, vector<double>(30, 1.4));

// 以下も可：3x3
vector<vector<int>> z {
  vector<int>(3, 1),
  vector(3, 1),
  { 1, 2, 3 }
};
```

1次元配列と2次元配列の初期化の比較（図）

整数型の配列

int型の配列 std::vector<int>

	v[0]	v[1]	v[2]	v[3]	v[4]
v	2	3	5	7	11

整数型の配列の配列

std::vector<int>型の配列 std::vector<std::vector<int>>

	m[0]	m[1]	m[2]	m[3]	m[4]
m	2 3 5	7 11 13	17 19 23	29 31 37	41 43 47

整数型の配列の配列（見やすく）

std::vector<int>型の配列
std::vector<std::vector<int>>

	m[x][0]	m[x][1]	m[x][2]
m[0]	2	3	5
m[1]	7	11	13
m[2]	17	19	23
m[3]	29	31	37
m[4]	41	43	47

3次元配列の初期化

- 4x5x2の例

```
// 要素はすべて0
vector xx( 4, vector( 5, vector(2, 0) ) );
xx[0][0][0] = 10;

// それぞれの値を持つ場合
vector<vector<vector<int>>> xxa {
  {{1,2}, {2,3}, {3,4}, {5,6}, {6,7}},
  {{2,7}, {3,8}, {4,2}, {6,9}, {7,3}},
  {{3,4}, {4,2}, {5,1}, {7,3}, {8,8}},
  {{7,3}, {0,8}, {9,6}, {1,4}, {7,2}}
};
cout << xxa[0][0][0]; // 1 先頭の要素
cout << xxa[0][0][1]; // 2
// cout << xxa[0][0][2]; // この要素番号はない
cout << xxa[0][1][0]; // 2
cout << xxa[3][4][1]; // 2 一番最後の要素
```

2次元配列とループ

- `a.size()` と `a[i].size()` で要素数を確認
  - `a` は `vector<double>` を要素として持つ `vector`
  - `a[i]` は `double` を要素として持つ `vector`
- `size_t` 型 (0以上の整数型) の利用
  - `size_t` : 配列のインデックスなどを表すために設計された型．非負．範囲が環境に依存しない．
  - `int` : 整数いろいろに使える型．負の範囲がある．範囲が環境に依存．配列のインデックスには不適．
- `.size()` の戻り値の型は `size_t` ．暗黙的キャストを避けるため， `size_t` を使うとよい．

```cpp
int main() {
  vector<vector<double>> a {
    {2.3, 8.4, 4.3, 1.2},
    {8.2, 1.3, 8.1, 7.5},
    {3.3, 3.1, 9.8, 5.3}
  };

  std::cout << a.size() << ", "
            << a[0].size() << "\n";

  for (size_t i = 0; i < a.size(); i++) {
    for (size_t j = 0; j < a[i].size(); j++)
      cout << a[i][j] <<" ";
    cout <<"\n";
  }
}
```

```
% ./a.out
3, 4
2.3 8.4 4.3 1.2
8.2 1.3 8.1 7.5
3.3 3.1 9.8 5.3
```

2次元配列の入出力の例

- 要素数が事前に分かる場合は定数を用いる
- `for` 文の添字は `int` でも良い(`.size()` を使わないため)

```cpp
int main() {
  const int n{3}, m{4};
  vector b(n, vector(m, 0));

  // // 以下のように書くのは非合理なので for文 を使いましょう
  // std::cin >> b[0][0] >> b[0][1] >> b[0][2] >> b[0][3]
  //          >> b[1][0] >> b[1][1] >> b[1][2] >> b[1][3]
  //          >> b[2][0] >> b[2][1] >> b[2][2] >> b[2][3];

  for (int i = 0; i < n; i++)
    for (int j = 0; j < m; j++)
      std::cin >> b[i][j];

  for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {
      if (j != 0) cout <<", ";
      cout << b[i][j];
    }
    cout <<"\n";
  }
}
```

```
./a.out
1 2 3 4 5 6 7 8 9 10 11 12
1, 2, 3, 4
5, 6, 7, 8
9, 10, 11, 12
```

範囲 `for` 文の利用

- 添字情報が不要なら場合には記述が簡潔になる
- リファレンスの利用が性能に影響を与える
- 2次元配列 `a` の中から1次元配列を `x` に一つずつ取り出しながら繰り返し、内側のループでは `x` の中から `double` 型の要素を `y` に取り出して出力

```cpp
int main() {
  vector<vector<double>> a{
    {2.3, 8.4, 4.3, 1.2},
    {8.2, 1.3, 8.1, 7.5},
    {3.3, 3.1, 9.8, 5.3}
  };

  // 範囲 for 文の利用
  for (const vector<double>& x : a) {
    for (double y : x)
      cout << y << " ";
    cout << "\n";
  }

  // これは上と同じ挙動
  for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 4; j++)
      cout << a[i][j] << " ";
    cout << "\n";
  }
}
```

`auto` の利用

- 初期値から型を推定する
- 複雑な型名に有効
  - 単純な型には使用しない方が良い
- 型指定の面倒さは `auto` を使って解決できる
  - が，`auto` の乱用はプログラムを読みにくくする

```
./a.out
1 2.1

1 1 1

2.1 2.1 2.1 2.1
2.1 2.1 2.1 2.1
2.1 2.1 2.1 2.1
```

```cpp
int main() {
  auto i { 1 };    // int
  auto d { 2.1 };  // double
  cout << i << " " << d << "\n\n";

  auto v { vector(3, 1) };  // vector<int>
  for (auto x : v)          // int
    cout << x << " ";
  cout << "\n";

  // vector<vector<double>>
  auto t { vector(3, vector(4, 2.1)) };

  for (const auto& x : t) { // const vector<double>&
    for (auto y : x)  // double
      cout << y << " ";
    cout << "\n";
  }
}
```

3次元配列とループ

- 各次元の要素数に注意
- 添字の変数名には `i, j, k` を用いる習慣がある

```
vector<vector<vector<int>>> a {
  {{0,-1,2,3}, {-4,5,6,7},  {8,-9,10,11}},
  {{0,1,-2,3}, { 4,5,-6,7}, {8,9,-10,11}}
};

// 負の要素を数える
int n {0};
for (size_t i = 0; i < a.size(); i++)
  for (size_t j = 0; j < a[i].size(); j++)
    for (size_t k = 0; k < a[i][j].size(); k++)
      if (a[i][j][k] < 0) ++n;
std::cout << n << "\n"; // 6
```

3次元配列と範囲for文

- 添字は不要だが各次元用の変数名が必要
- リファレンス指定の有無が性能に影響する

```
vector<vector<vector<int>>> a {
  {{0,-1,2,3}, {-4,5,6,7},  {8,-9,10,11}},
  {{0,1,-2,3}, { 4,5,-6,7}, {8,9,-10,11}}
};

n = 0;
for (const auto& x : a)
  for (const auto& y : x)
    for (auto z : y)
      if (z < 0) ++ n;
std::cout << n <<"\n"; // 6
```

n x m行列と転置行列

- 入力はn → mの順，出力はm → nの順
- 添字の変化に注意

```
% ./a.out
1 2 3 4 5 6 7 8 9 10 11 12

1 2 3 4
5 6 7 8
9 10 11 12

1 5 9
2 6 10
3 7 11
4 8 12
```

```cpp
int main() {
  const int n{3}, m{4};
  auto data { vector(n, vector(m, 0)) }; // vector<vector<int>>

  // n x m の 2 次元配列に入力
  for (int i = 0; i < n; i++) // 行方向
    for (int j = 0; j < m; j++) // 列方向
      std::cin >> data[i][j];
  cout <<"\n";

  // n x m の行列を出力
  for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++)
      cout << data[i][j] <<" ";
    cout <<"\n";
  }
  cout <<"\n";

  // m x n の転置行列を出力
  for (int j = 0; j < m; j++) { // 転置の行方向
    for (int i = 0; i < n; i++) // 転置の列方向
      cout << data[i][j] <<" ";
    cout <<"\n";
  }
  cout <<"\n";
}
```

2x2の逆行列

- `abs()` には `<cmath>` が必要
- `ia` の宣言に注意（空の `vector<double>` が2個）
- `ia[0] = { 値1, 値2 };`
  - `=` の左辺 `ia[0]` は `vector<double>` なので，右辺を `{ 値1, 値2 }` とすると，`ia[0][0]` が `値1`，`ia[0][1]` が `値2` になる

```
% ./a.out
1 3
5 7

-0.875 0.375
0.625 -0.125
```

$$A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}, A^{-1} = \frac{1}{ad - bc} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$$

```cpp
int main() {
  vector<vector<double>> a { {1.0,3.0}, {5.0,7.0} };

  // a の逆行列が存在するかを判定
  const double epsilon {1.0e-8};
  const double det {
    a[0][0] * a[1][1] - a[0][1] * a[1][0]
  };

  // double型 == 0 の比較を避ける
  if (abs(det) < epsilon) {
    cout <<"no inverse\n"; return 1;
  }

  // a の逆行列を計算
  vector<vector<double>> ia(2);
  ia[0] = { a[1][1]/det, -a[0][1]/det };
  ia[1] = {-a[1][0]/det, a[0][0]/det };

  cout << ia[0][0] <<" "<< ia[0][1] <<"\n"
       << ia[1][0] <<" "<< ia[1][1] <<"\n\n";
}
```

2次元配列の引数

- `vector` 配列は何次元でも引数として指定できる
- 基本は値渡し（要素のコピー）なので参照渡しが効率的
  - `const vector<T>&` または `vector<T>&`
- ※ 参照渡し（リファレンス）については第5回関数について参照
- ※ 関数呼び出しの実引数の型と，関数の宣言における仮引数の型を意識する

```
% ./a.out
1 2 3 4 5
2 3 4 5 6
3 4 5 6 7

2 3 4 5 6
3 4 5 6 7
4 5 6 7 8
```

```cpp
// 二次元配列の各要素を1増やす
// &による参照渡し（速度面で有利）
void incr( vector<vector<int>>& x ) {
  for( vector<int>& v : x )
    for( int& i : v ) i++;
}

// 二次元配列の各要素を表示
// &による参照渡し，constによる変更不可で安全
void print( const vector<vector<int>>& x ){
  for( const vector<int>& v : x )
    for( int i : v ) std::cout << i << " ";
  std::cout << "\n";
}

int main() {
   vector<vector<int>> a {
    {1,2,3,4,5},
    {2,3,4,5,6},
    {3,4,5,6,7}
  };

  print(a);
  incr(a);
  print(a);
}
```

2次元配列の一部を実引数にする例

- 2次元配列は配列の配列である点を利用

```
% ./a.out
2 3 4 5 6
```

```
1  // int の 1 次元配列を出力する
2  void print_int_array(const vector<int>& x) {
3    for (size_t i = 0; i < x.size(); i++) {
4      if (i != 0) cout << " ";
5      cout << x[i];
6    }
7    cout <<"\n";
8  }
9
10 int main() {
11   vector<vector<int>> a {
12     {1,2,3,4,5},
13     {2,3,4,5,6},
14     {3,4,5,6,7}
15   };
16   print_int_array( a[1] );
17 }
```

string配列との類似性

- `string` は1次元配列と同じ扱い
- `string` の `vector` 配列は2次元配列に見える

```
% ./a.out
e 2 g
```

```
// 文字列を 1 文字ずつ間隔をあけて出力する
void print_capital(string s) {
  for (size_t i = 0; i < s.size(); i++) {
    if (i != 0) cout << " ";
    cout << s[i];
  }
  cout <<"\n";
}

int main() {
  vector<string> b {"a1cd", "e2g", "hi"};
  print_capital( b[1] );
}
```

配列を返す関数

- 基本としてはすべての要素をコピーで返す
- 返す配列が局所変数の場合はコピーをせず再利用

```
% ./a.out
3 2
10 11 20 21 30 31
10 11
20 21
30 3
```

```cpp
// 一次元配列を返す例
vector<int> input(int n) {
  vector<int> v(n);
  for (auto& e: v) std::cin >> e;
  return v;
}

int main() {
  int row{}, col{};
  std::cin >> row >> col; // n x mを入力

  vector<vector<int>> t(row);
  for (auto& e: t) // 2 次元配列の入力
    e = input(col);

  for (const vector<int>& x : t) {
    for (int y : x) cout << y << " ";
    cout << "\n";
  }
}
```

```cpp
// 二次元配列を返す例
// このコードは左のコードと同じ挙動
vector<vector<int>> input2d(int n, int m) {
  vector<vector<int>> mat { vector(n, vector(m, 0) ) };
  for (auto& v: mat)
    for (auto& e: v)
      std::cin >> e;
  return mat;
}

int main() {
  int row{}, col{};
  std::cin >> row >> col; // n x mを入力

  vector<vector<int>> t;
  t = input2d( row, col );

  for (const vector<int>& x : t) {
    for (int y : x) cout << y << " ";
    cout << "\n";
  }
}
```